



PERTEMUAN I

NUMPY DAN PANDAS

1.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar Numpy dan penggunaan *array* n-dimensi serta berbagai operasi manipulasi *array*
- B. Mahasiswa mengetahui berbagai manipulasi dan transformasi data pada pandas, termasuk pembuatan DataFrame
- C. Mahasiswa dapat menerapkan Numpy dan pandas untuk mengolah dan menganalisis data

1.2. DASAR TEORI

- **Numpy**

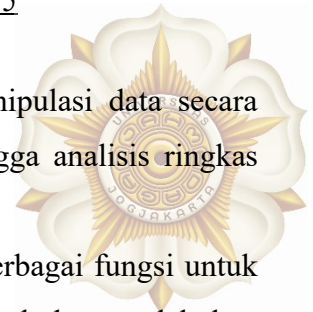
NumPy (Numerical Python) adalah pustaka Python yang digunakan untuk membangun dan mengelola *array* sebagai dasar komputasi numerik. NumPy dikembangkan pada tahun 2005 oleh Travis Oliphant sebagai proyek *open-source*. NumPy menawarkan kecepatan pemrosesan yang lebih tinggi, penggunaan memori yang efisien, serta dukungan terhadap struktur *array* berdimensi banyak. Kemampuan ini menjadikan NumPy penting dalam analisis numerik dan komputasi ilmiah.

Struktur utama NumPy adalah *ndarray*, yaitu *array* yang dapat berbentuk satu atau lebih dimensi, mulai dari 1D hingga 3D dan seterusnya. *Array* dapat dibentuk dari berbagai struktur sekuensial di Python maupun melalui fungsi internal untuk menghasilkan nilai tertentu.

Akses elemen pada *array* dilakukan melalui *indexing*, yaitu penunjukan posisi berdasarkan indeks yang dimulai dari nol. NumPy juga mendukung *slicing*, yakni pengambilan sebagian elemen berdasarkan rentang indeks tertentu, termasuk pada *array* multidimensi. Mekanisme ini memungkinkan manipulasi data secara efisien tanpa menyalin seluruh *array*.

- **Pandas**

Pandas adalah pustaka Python yang dirancang untuk mempermudah pengolahan dan analisis data terstruktur. Pustaka ini menyediakan dua struktur data utama, yaitu Series (data satu dimensi) dan DataFrame (data dua dimensi berbentuk tabel). Dengan indeks



yang fleksibel, Pandas memungkinkan pengguna melakukan manipulasi data secara efisien, mencakup pembersihan, transformasi, penggabungan, hingga analisis ringkas terhadap dataset.

Sebagai *library* utama dalam data analisis, Pandas menyediakan berbagai fungsi untuk memodifikasi struktur DataFrame, seperti menambah atau menghapus kolom, melakukan pengurutan, memfilter baris berdasarkan kondisi, serta melakukan *grouping* dan *aggregation* untuk merangkum data. Selain itu, Pandas juga mendukung operasi lanjutan seperti *merging*, *joining*, *concatenation*, dan *reshaping* melalui *pivoting* atau *melting*, yang menjadikannya sangat fleksibel untuk mengelola dataset dalam berbagai format.

Pandas dirancang dengan integrasi yang baik bersama NumPy, sehingga operasi numerik dan manipulasi *array* dapat dilakukan secara konsisten pada struktur DataFrame. Dengan performa yang efisien dan sintaks yang mudah dibaca, Pandas menjadi salah satu pustaka paling penting dalam ekosistem *data science* untuk penanganan data tabular.

1.3. ALAT DAN BAHAN

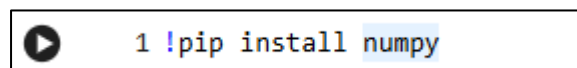
- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colaboratory

1.4. LANGKAH PERCOBAAN

- **NumPy**

- 1) Install NumPy

NumPy telah terinstal di Google Colaboratory secara *default*. Jika belum terinstal, instalasi numpy dapat dilakukan dengan menggunakan PIP. Pada Jupyter Notebook termasuk Google Colab, instalasi NumPy dapat dilakukan seperti Gambar 1.4.1.

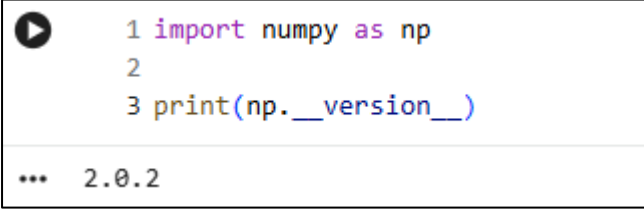


Gambar 1.4.1 Instalasi NumPy

- 2) Impor pustaka NumPy

Impor pustaka dengan *syntax* `import <pustaka>`. Nama alias juga dapat diberikan untuk mempersingkat pengetikan nama pemanggilan pustaka. Secara umum alias np

digunakan untuk menyingkat numpy. Melakukan impor dan membuat alias dapat dilakukan seperti Gambar 1.4.2. Pada waktu pembuatan modul ini, pustaka NumPy yang terinstal di Google Colaboratory adalah pustaka NumPy versi 2.0.2.



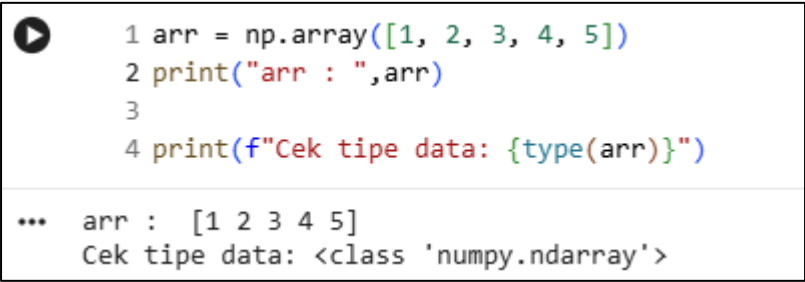
```
1 import numpy as np
2
3 print(np.__version__)

... 2.0.2
```

Gambar 1.4.2 Impor pustaka NumPy

3) Membuat *array* 1 dimensi

Membuat *array* dapat dilakukan dengan memanggil fungsi *array* seperti Gambar 1.4.3. Tipe data numpy berupa *ndarray* yang memiliki arti N-dimensional *array*.



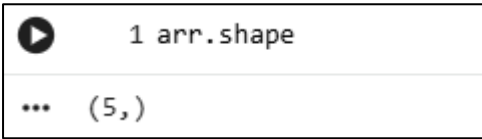
```
1 arr = np.array([1, 2, 3, 4, 5])
2 print("arr : ",arr)
3
4 print(f"Cek tipe data: {type(arr)}")

... arr : [1 2 3 4 5]
   Cek tipe data: <class 'numpy.ndarray'>
```

Gambar 1.4.3 Membuat *array* 1 dimensi

4) Melihat *shape* atau bentuk *array*

Untuk melihat bentuk *array* yang telah dibuat, dapat dilakukan dengan memanggil atribut *shape* pada objek *ndarray*. Lakukan seperti Gambar 1.4.4, hasil tersebut menginformasikan bahwa *array* terdiri dari 1 dimensi dan terdiri dari 5 elemen.



```
1 arr.shape

... (5,)
```

Gambar 1.4.4 Melihat bentuk *array*

5) Membuat *array* 2 dimensi

Membuat *array* 2 dimensi dapat dilakukan pada fungsi yang sama. Untuk membuat *array* 2 dimensi, masukkan objek *list* seperti pembuatan dimensi 1 ke dalam *list* sehingga membentuk *nested list* seperti pada Gambar 1.4.5. Jika di cek bentuk *array*-nya maka akan menampilkan (2, 3). Angka dua ini menunjukkan banyak elemen di

sumbu 0 atau di *list* terluar, sementara itu angka tiga menunjukkan banyak elemen di sumbu 1 atau *list* bagian dalam.



```
1 arr2d = np.array([[1,2,3],[4,5,6]])
2 print(arr2d)
3
4 print(f"Shape: {arr2d.shape}")
5 # Angka pertama = member di sumbu 0,
6 # Angka kedua =member di sumbu 1
```

```
... [[1 2 3]
      [4 5 6]]
      Shape: (2, 3)
```

Gambar 1.4.5 Membuat array 2 dimensi dan mengecek bentuk array

6) Membuat *array* 3 dimensi

Serupa dengan konsep pembuatan *array* 2 dimensi, *array* 3 dimensi akan memuat elemen 2 dimensi. Contoh pembuatan *array* 3 dimensi dapat dilihat di Gambar 1.4.1.



```

1 arr3d = np.array(
2     [
3         [
4             [1, 2, 3, 4], [5, 6, 7, 8]
5         ],
6         [[9,10 ,11, 12], [13, 14, 15, 16]],
7         [[16, 17, 18, 19], [20, 21, 22, 23]]
8     ]
9 )
10
11 print(arr3d)
12
13 print(f"Shape: {arr3d.shape}")
14 # Angka pertama = member di sumbu 0,
15 # Angka kedua = member di sumbu 1,
16 # Angka ketiga = member di sumbu 2

```

```

... [[[ 1  2  3  4]
      [ 5  6  7  8]]

      [[ 9 10 11 12]
      [13 14 15 16]]

      [[16 17 18 19]
      [20 21 22 23]]]
Shape: (3, 2, 4)

```

Gambar 1.4.6 Membuat array 3 dimensi

7) Membuat *array* dengan *String* sebagai member

```

1 arr_str = np.array(['hello', 'nama', 'saya'])
2 print(arr_str)

```

```

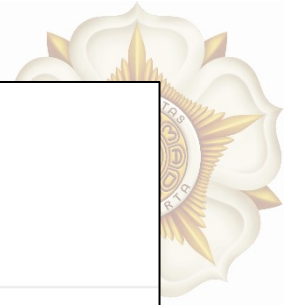
... ['hello' 'nama' 'saya']

```

Gambar 1.4.7 Array String

8) Melakukan *indexing* di *array* 1 dimensi

Indexing tidak jauh berbeda dengan fungsi *native indexing* untuk tipe data *list*. Mengakses elemen dengan indeks ke *n* dapat dilakukan dengan `objek_ndarray[n]` seperti Gambar 1.4.8 baris ke 3. Perlu diingat bahwa indeks selalu mulai dari 0.



```
1 arr = np.array([1,2,3,4,5])
2 # menampilkan elemen ke tiga
3 print(arr[2])
4 # hasil penjumlahan elemen 1 dengan elemen 5
5 print(arr[0] + arr[4])

... 3
     6
```

Gambar 1.4.8 Indexing array 1 dimensi

9) Melakukan *indexing* di array 2 dimensi

```
1 arr = np.array([[1, 2, 3], [4, 5, 6]])
2 # baris pertama, kolom ketiga. atau elemen 1 di sumbu 0, elemen 3 di sumbu 1
3 print(arr[0][2])
4 # baris kedua, kolom kedua. atau elemen 2 sumbu 0, elemen 2 sumbu 1
5 print(arr[1][1])

... 3
     5
```

Gambar 1.4.9 Indexing array 2 dimensi

10) Melakukan *indexing* di array 3 dimensi

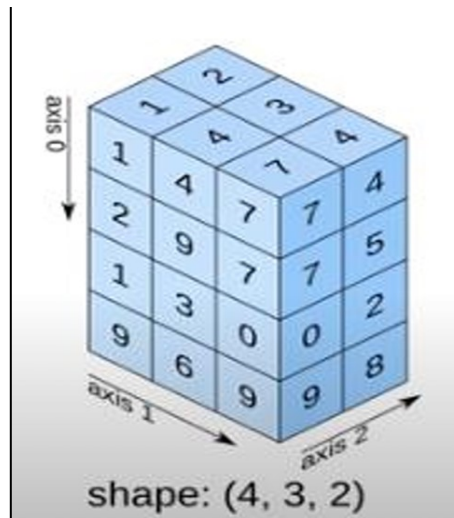
```
1 arr = np.array([
2     [[1, 2, 3], [4, 5, 6]],
3     [[10, 11, 12], [13, 14, 14]]
4 ])
5
6 # menampilkan angka 11
7 #sumbu 0 : index 1
8 #sumbu 1 : index 0
9 #sumbu 2 : index 1
10 print(arr[1][0][1])
11
12 # menampilkan angka 14
13 #sumbu 0 : index 1
14 #sumbu 1 : index 1
15 #sumbu 2 : index 2
16 print(arr[1][1][2])

... 11
     14
```

Gambar 1.4.10 Indexing array 3 dimensi

11) Membuat *array* dari contoh

Diberikan sebuah *array* dengan bentuk seperti Gambar 1.4.11. Bagian yang tidak terlihat dianggap memiliki nilai 0.



Gambar 1.4.11 Contoh array 3 dimensi

Jika Anda diminta untuk membuatnya dalam bentuk kode, maka akan terlihat seperti Gambar 1.4.12.

```

1 implementasi_array_3d = np.array([
2     [[1, 2], [4, 3], [7, 4]],
3     [[2, 0], [9, 0], [7, 5]],
4     [[1, 0], [3, 0], [0, 2]],
5     [[9, 0], [6, 0], [9, 8]]
6 ])
7 print(implementasi_array_3d)
8
9 print("\nshape", implementasi_array_3d.shape)
10 # sumbu 0 : 4 elemen
11 # sumbu 1 : 3 elemen
12 # sumbu 2 : 2 elemen
13 # silahkan cek di slide halaman 5

... [[1 2]
      [4 3]
      [7 4]]

      [[2 0]
      [9 0]
      [7 5]]

      [[1 0]
      [3 0]
      [0 2]]

      [[9 0]
      [6 0]
      [9 8]]]

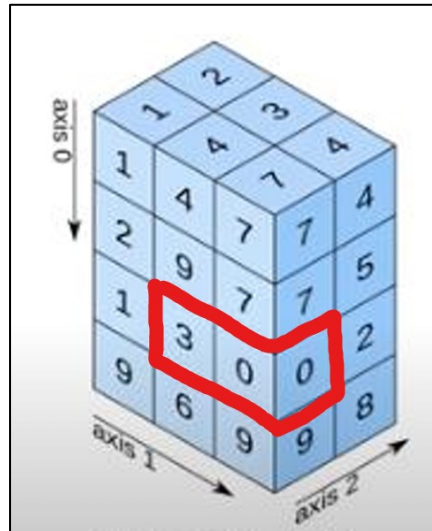
shape (4, 3, 2)

```

Gambar 1.4.12 Implementasi Array

12) Array slicing

NumPy menerapkan cara yang sama dengan cara *slicing* tipe data List untuk melakukan *slicing* tipe data *ndarray*. Sebagai contoh Anda ingin untuk mengambil Gambar 1.4.13 yang diberi tanda merah (3 dan 0), maka Anda harus bisa menentukan indeks lokasi target tersebut.



Gambar 1.4.13 Target slicing

Di sumbu 0, target tersebut berada di indeks ke 2. Di sumbu 1, target berada di indeks ke 1 dan 2 atau dapat disebut indeks ke 1 dan selanjutnya. Di sumbu 2, target hanya di indeks ke 0. Dengan demikian, penulisan kode *slicing* dapat dilakukan seperti Gambar 1.4.14.

```
1 target = implementasi_array_3d[2][1:][0]
2 print(target)

... [3 0]
```

Gambar 1.4.14 Contoh penerapan Slicing

13) Mengonversi tipe data

Melakukan konversi tipe data dapat dilakukan dengan fungsi **astype** dengan memasukkan target kelas.



```
1 arr = np.array(['1', '2', '3', '4', '5'])
2 print(arr)
3
4 arr_int = arr.astype(int)
5 print("Integer", arr_int)
6
7 arr_float = arr.astype(float)
8 print("Float", arr_float)

... ['1' '2' '3' '4' '5']
Integer [1 2 3 4 5]
Float [1. 2. 3. 4. 5.]
```

Gambar 1.4.15 Konversi tipe data di NumPy

14) *Reshape* ke dimensi lebih tinggi

Untuk mengubah bentuk *array* ke dimensi lebih tinggi, pastikan bahwa banyak elemen tetap harus sama. Sebagai contoh Gambar 1.4.16, banyak elemen *array* asli adalah 12, ketika ingin mengubah bentuk ke 2 dimensi, total banyak elemen tetap harus 12. Sebagai contoh perhitungannya, *reshaping* ke dimensi 2 dilakukan dengan banyak elemen di sumbu 0 adalah 3 dan di sumbu 1 adalah 4, total dari banyak elemen tersebut adalah 12 (cara perhitungan : $3 \times 4 = 12$). Hasil tersebut tetap sama dengan banyak elemen di *array* asli.



```
1 arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
2 print(arr)
3 print('Shape', arr.shape, '\n')
4
5 new_arr_2d = arr.reshape(3, 4)
6 # sumbu 0 : 3 member
7 # sumbu 1 : 4 member
8 print(new_arr_2d)
9 print('Shape', new_arr_2d.shape, '\n')
10
11 new_arr_3d = arr.reshape(3, 2, 2)
12 # sumbu 0 : 3 member
13 # sumbu 1 : 2 member
14 # sumbu 2 : 2 member
15 print(new_arr_3d)
16 print('Shape', new_arr_3d.shape)
17
```

```
... [ 1  2  3  4  5  6  7  8  9 10 11 12]
Shape (12,)
```

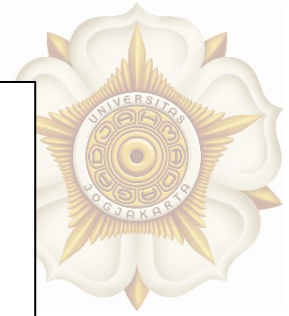
```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape (3, 4)
```

```
[[[ 1  2]
   [ 3  4]]
 [[ 5  6]
   [ 7  8]]
 [[ 9 10]
   [11 12]]]
Shape (3, 2, 2)
```

Gambar 1.4.16 Contoh Reshape ke dimensi lebih tinggi

15) *Reshape* ke dimensi 1

Untuk melakukan *reshape* ke dimensi yang lebih rendah, Anda perlu melakukan *reshape* ke dimensi 1 terlebih dahulu. Untuk mengubah bentuk ke dimensi 1 dapat dilakukan seperti Gambar 1.4.17. Setelah melakukan pengubahan bentuk ke dimensi 1, perubahan ke dimensi ke lebih tinggi dapat dilakukan.



```

1 #3d ke 1 d
2 arr = np.array([
3     [[1, 2, 3], [4, 5, 6]],
4     [[10, 11, 12], [13, 14, 15]]
5 ])
6 print(arr)
7 print('sesudah di reshape')
8 new_arr = arr.reshape(-1)
9
10 print(new_arr)

```

```

...  [[ 1  2  3]
      [ 4  5  6]]

      [[10 11 12]
       [13 14 15]]]
sesudah di reshape
[ 1  2  3  4  5  6 10 11 12 13 14 15]

```

Gambar 1.4.17 Reshape ke dimensi 1

16) Perbedaan fungsi *len* dan atribut *shape*

Telah kita mengetahui bahwa *shape* akan menampilkan bentuk *array*, sementara itu, *len* hanya mampu menghitung banyak elemen yang ada di sumbu 0. Masih menggunakan Gambar 1.4.11, berikut akan membandingkan hasil perbedaan *len* dan *shape*. Di hasil Gambar 1.4.18, terlihat bahwa *len* hanya memunculkan angka 4 karena di sumbu 0 hanya ada 4 elemen.

```

1 print(len(implementasi_array_3d))
2 print(implementasi_array_3d.shape)

```

```

...  4
      (4, 3, 2)

```

Gambar 1.4.18 Len vs Shape

17) Iterasi *ndarray*

```

1 for sumbu_0 in implementasi_array_3d:
2     for sumbu_1 in sumbu_0:
3         for sumbu_2 in sumbu_1:
4             print(sumbu_2, end=", ")
5

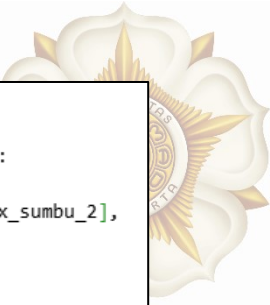
```

```

...  1, 2, 4, 3, 7, 4, 2, 0, 9, 0, 7, 5, 1, 0, 3, 0, 0, 2, 9, 0, 6, 0, 9, 8,

```

Gambar 1.4.19 Iterasi langsung



```

1 for idx_sumbu_0 in range(implementasi_array_3d.shape[0]):
2     for idx_sumbu_1 in range(implementasi_array_3d.shape[1]):
3         for idx_sumbu_2 in range(implementasi_array_3d.shape[2]):
4             print(
5                 implementasi_array_3d[idx_sumbu_0][idx_sumbu_1][idx_sumbu_2],
6                 end=", "
7             )
8
... 1, 2, 4, 3, 7, 4, 2, 0, 9, 0, 7, 5, 1, 0, 3, 0, 0, 2, 9, 0, 6, 0, 9, 8,

```

Gambar 1.4.20 Iterasi dengan indeks

18) Melakukan *join* pada *array* dimensi 1

Menggabungkan *array* dapat dilakukan dengan fungsi *concatenate*. Untuk menggabungkan *array* berdimensi 1 maka sumbu (*axis*) di set ke 0 seperti Gambar 1.4.21.

```

1 arr1 = np.array([1, 2, 3])
2 arr2 = np.array([4, 5, 6])
3 arr3 = np.concatenate((arr1, arr2), axis=0)
4 print(arr3)
... [1 2 3 4 5 6]

```

Gambar 1.4.21 Concatenate 2 array

19) Melakukan *join* pada *array* dimensi 2

Di dimensi 2, Anda bisa memilih untuk menggabungkan *array* ke sumbu 1 atau 2. Penggabungan *array* ke sumbu 1 akan terlihat seperti Gambar 1.4.22. Penggabungan *array* ke sumbu 0 akan terlihat seperti Gambar 1.4.23.

```

1 arr1 = np.array([[1, 2, 3], [4, 5, 6]])
2 arr2 = np.array([[11, 12, 13], [14, 15, 16]])
3
4 # kita menambah di sumbu 1, kolomnya
5 arr3 = np.concatenate((arr1, arr2), axis=1)
6 print(arr3)
... [[ 1  2  3 11 12 13]
     [ 4  5  6 14 15 16]]

```

Gambar 1.4.22 Menambahkan Array di sumbu 1



```

1 # kita menambah di sumbu 0, baris nya
2 arr3 = np.concatenate((arr1, arr2), axis=0)
3 print(arr3)

... [[ 1  2  3]
      [ 4  5  6]
      [11 12 13]
      [14 15 16]]

```

Gambar 1.4.23 Menambahkan Array di sumbu 0

20) Array Split

Melakukan *split* pada objek *ndarray* dapat dilakukan dengan fungsi **array_split** seperti Gambar 1.4.24.

```

1 arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
2 #dipisah jadi 5 bagian array
3 newarr = np.array_split(arr, 5)
4 print('sebelum')
5 print(arr)
6 print('sesudah')
7 print(newarr)
8 narr = newarr[2]
9 print(narr)

... sebelum
[ 1  2  3  4  5  6  7  8  9 10 11 12]
sesudah
[array([1, 2, 3]), array([4, 5, 6]), array([7, 8]), array([ 9, 10]), array([11, 12])]
[7 8]

```

Gambar 1.4.24 Array Split

21) Array Search

Melakukan pencarian dapat dilakukan dengan fungsi **where** dengan memasukkan kondisi seperti Gambar 1.4.25.

```

1 arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
2 #mencari elemen yg nilainya 3, dan mengembalikan index nya
3 x = np.where(arr == 3 )
4 print(x)

... (array([2]),)

```

Gambar 1.4.25 Mencari elemen dalam array

22) Sorting

Melakukan *sorting* dapat dilakukan dengan fungsi **sort**. Terdapat beberapa parameter yang bisa digunakan. Penjelasan lengkap mengenai parameter *sorting* dapat dibaca di dokumentasi NumPy. Sebagai contoh Gambar 1.4.26, *sorting* dilakukan secara *ascending*.



```

1 arr = np.array([11,21,3,4,19,1,21,12])
2 new_arr = np.sort(arr)
3 print(arr)
4 print(new_arr)

... [11 21  3  4 19  1 21 12]
    [ 1  3  4 11 12 19 21 21]

```

Gambar 1.4.26 Sorting NumPy

23) NumPy Random

Impor *package* random dari pustaka numpy untuk menggunakan numpy random. Gunakan fungsi **rand** untuk membuat nilai *random* dengan *shape* n. Untuk membuat nilai random dengan *shape* dimensi 1 dan 2 elemen, maka akan terlihat seperti Gambar 1.4.27.

```

1 from numpy import random
2 x = random.rand(2)
3 print(x)

... [0.27114082 0.92494933]

```

Gambar 1.4.27 Numpy Random

24) Operasi penjumlahan

NumPy memiliki operasi penjumlahan *array*. Operasi tersebut dapat diakses dengan fungsi **add** seperti Gambar 1.4.28.

```

1 arr1 = np.array([1,2,3,4,5])
2 arr2 = np.array([10,20,30,40,50])
3 #elemen dari satu array ditambah dengan elemen di array lain
4 arr3 = np.add(arr1, arr2)
5 print(arr3)

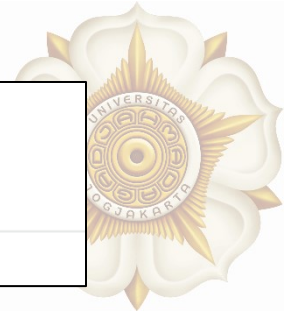
... [11 22 33 44 55]

```

Gambar 1.4.28 Penjumlahan Array

25) Operasi *Summation* (Total jumlah)

NumPy juga memberikan operasi untuk melakukan kalkulasi total jumlah semua elemen.



```
1 arr1 = np.array([1,2,3,4,5])
2 arr3 = np.sum(arr1)
3 print(arr3)
```

... 15

Gambar 1.4.29 Summation

26) Product of elements

Untuk menghitung perkalian semua elemen, dapat dilakukan dengan menggunakan fungsi **prod**.

```
1 arr1 = np.array([1,2,3,4,5])
2 arr3 = np.prod(arr1)
3 print(arr3)
```

... 120

Gambar 1.4.30 Product of elements NumPy

• Dataframe Pandas

1) Instal Pandas

Pandas telah terinstal di Google Colaboratory secara *default*. Jika belum terinstal, instalasi pandas dapat dilakukan dengan menggunakan PIP. Pada Jupyter Notebook termasuk Google Colaboratory, instalasi Pandas dapat dilakukan seperti Gambar 1.4.31.

```
!pip install pandas
```

Gambar 1.4.31 Instalasi pandas

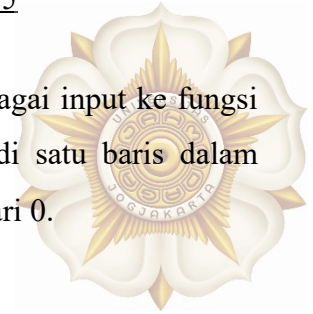
2) Import Library

Import library dengan *syntax* **import <pustaka>**. Nama alias juga dapat diberikan untuk mempersingkat pengetikan nama pemanggilan pustaka. Secara umum alias **pd** digunakan untuk menyingkat **pandas**. Pada percobaan ini, kita memerlukan *library* **pandas** dan **numpy**, sehingga kita akan mengimpor kedua *library* tersebut.

```
import pandas as pd
import numpy as np
```

Gambar 1.4.32 Import Library pandas

3) Membuat Dataframe



DataFrame dibuat dengan memberikan sebuah list Python sebagai input ke fungsi `pd.DataFrame()`. Setiap elemen dalam list otomatis menjadi satu baris dalam *DataFrame*, sementara indeks dihasilkan secara default mulai dari 0.

```
data = [2, 3, 5, 7, 1]
df = pd.DataFrame(data)
df
```

	0
0	2
1	3
2	5
3	7
4	1

Gambar 1.4.33 Membuat Dataframe

Kita juga dapat menambahkan parameter *columns* untuk menetapkan nama kolom dan parameter *index* untuk memberikan label indeks *custom*. Hasilnya, *DataFrame* tampil dalam format tabular seperti pada Gambar 1.4.34.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'], index=['satu', 'dua', 'tiga'])
df
```

	Nama	Umur
satu	Adi	34
dua	Maya	23
tiga	Joko	20

Gambar 1.4.34 Membuat Dataframe

4) Menambahkan Kolom

Penambahan kolom pada data *frame* dapat dilakukan dengan menambahkan `df['Nama Kolom']`.



```
data = {'Nama': ['Jack', 'Joko'], 'Umur': [20, 23]}
df = pd.DataFrame(data)
df['Alamat'] = ['Surabaya', 'Yogyakarta']
df
```

	Nama	Umur	Alamat
0	Jack	20	Surabaya
1	Joko	23	Yogyakarta

Gambar 1.4.35 Menambahkan Kolom

Penambahan kolom juga dapat dilakukan dengan fungsi `df.insert` seperti pada Gambar 1.4.36.

```
data = {'Nama': ['Jack', 'Joko'], 'Umur': [20, 23]}
df = pd.DataFrame(data)
print(df)
alamat = ['Medan', 'Jakarta']
df.insert(0, 'Alamat', alamat)
print(df)
```

	Nama	Umur
0	Jack	20
1	Joko	23

	Alamat	Nama	Umur
0	Medan	Jack	20
1	Jakarta	Joko	23

Gambar 1.4.36 Menambahkan Kolom

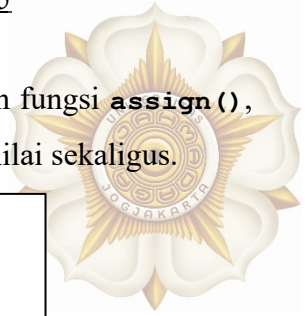
Kolom juga dapat ditambahkan pada posisi tertentu dengan menambahkan parameter `loc`, `column`, `value` pada fungsi `df.insert`.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
alamat = ['Medan', 'Jakarta', 'Solo']
df.insert(1, 'Alamat', alamat)
print(df)
```

	Nama	Alamat	Umur
0	Adi	Medan	34
1	Maya	Jakarta	23
2	Joko	Solo	20

Gambar 1.4.37 Menambahkan Kolom

5) Menambahkan Multi Kolom



Penambahan multi kolom dapat dilakukan dengan menggunakan fungsi `assign()`, yaitu dengan memberikan beberapa pasangan nama kolom dan nilai sekaligus.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
print(df)
alamat = ['Surabaya', 'Jakarta', 'Solo']
pekerjaan = ['Programmer', 'DB Admin', 'Web Developer']
df = df.assign(Job = pekerjaan, Alamat = alamat )
print('sesudah')
print(df)
```

	Nama	Umur
0	Adi	34
1	Maya	23
2	Joko	20

sesudah

	Nama	Umur	Job	Alamat
0	Adi	34	Programmer	Surabaya
1	Maya	23	DB Admin	Jakarta
2	Joko	20	Web Developer	Solo

Gambar 1.4.38 Menambahkan Multi Kolom

Penambahan multi kolom juga dapat dilakukan dengan membuat sebuah *dictionary* yang berisi pasangan *nama kolom* dan *nilai*, kemudian memasukkannya ke dalam *DataFrame* secara bersamaan.

```
data = [['Adi', 34], ['Maya', 23], ['Joko', 20]]
df = pd.DataFrame(data, columns = ['Nama', 'Umur'])
print(df)
alamat = ['Surabaya', 'Jakarta', 'Solo']
pekerjaan = ['Programmer', 'DB Admin', 'Web Developer']
d_kolom = {'Address': alamat, 'Job': pekerjaan}
df[['Address', 'Job']] = pd.DataFrame(d_kolom)
print(df)
```

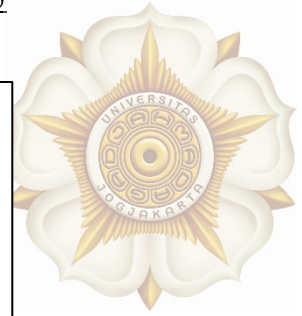
	Nama	Umur
0	Adi	34
1	Maya	23
2	Joko	20

	Nama	Umur	Address	Job
0	Adi	34	Surabaya	Programmer
1	Maya	23	Jakarta	DB Admin
2	Joko	20	Solo	Web Developer

Gambar 1.4.39 Menambahkan Multi Kolom

6) Menghapus Kolom

Penghapusan kolom dilakukan menggunakan fungsi `drop()` seperti pada Gambar 1.4.40. Parameter nama kolom dan *axis* digunakan untuk menentukan kolom yang akan dihapus. Parameter `inplace=True` membuat perubahan diterapkan langsung pada *DataFrame* tanpa membuat salinan baru.



```
data = {'Nama': ['Jack', 'Joko'],
        'Umur': [20, 23],
        'Alamat': ['Surabaya', 'Jakarta'],
        'Pekerjaan': ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
df.drop('Pekerjaan', axis=1, inplace=True)
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur	Alamat
0	Jack	20	Surabaya
1	Joko	23	Jakarta

Gambar 1.4.40 Menghapus Kolom

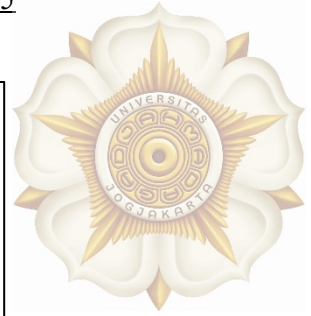
Penghapusan kolom juga dapat dilakukan dengan fungsi `df.pop()` (Gambar 1.4.41) dan juga perintah `del` (Gambar 1.4.42).

```
data = {'Nama': ['Jack', 'Joko'],
        'Umur': [20, 23],
        'Alamat': ['Surabaya', 'Jakarta'],
        'Pekerjaan': ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
df.pop('Alamat')
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur	Pekerjaan
0	Jack	20	Programmer
1	Joko	23	DB Admin

Gambar 1.4.41 Menghapus Kolom dengan Fungsi pop



```
data = {'Nama ': ['Jack', 'Joko'],
        'Umur' : [20, 23],
        'Alamat' : ['Surabaya', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
del df['Alamat']
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur	Pekerjaan
0	Jack	20	Programmer
1	Joko	23	DB Admin

Gambar 1.4.42 Menghapus Kolom dengan Perintah del

7) Menghapus Multi Kolom

Penghapusan multi kolom dapat dilakukan dengan fungsi `df.drop()` seperti gambar berikut.

```
data = {'Nama ': ['Jack', 'Joko'],
        'Umur' : [20, 23],
        'Alamat' : ['Surabaya', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin']}
df = pd.DataFrame(data)
print(df)
df.drop(['Alamat', 'Pekerjaan'], axis =1, inplace=True)
df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin

	Nama	Umur
0	Jack	20
1	Joko	23

Gambar 1.4.43 Menghapus Multi Kolom

8) Sorting

Mengurutkan data berdasarkan indeks pada *dataframe* dapat dilakukan dengan fungsi `df.sort_index()` seperti pada Gambar 1.4.45.

```
data = {'Nama ': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data, index=[2, 3, 0, 1])
print(df)
```

	Nama	Umur	Alamat	Pekerjaan
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer

Gambar 1.4.44 Membuat Dataframe

```
sorted_df = df.sort_index()
sorted_df
```

	Nama	Umur	Alamat	Pekerjaan
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin

Gambar 1.4.45 Sorting Data

Parameter **axis=1** digunakan untuk mengurutkan kolom berdasarkan indeks kolom.

```
sorted_column = sorted_df.sort_index(axis=1)
sorted_column
```

	Alamat	Nama	Pekerjaan	Umur
0	Solo	Dwi	Web developer	34
1	Jakarta	Andi	Programmer	23
2	Surabaya	Jack	Programmer	20
3	Jakarta	Joko	DB Admin	23

Gambar 1.4.46 Sorting Data

9) Sorting Value

Untuk mengurutkan data berdasarkan nilai pada kolom tertentu, digunakan fungsi **sort_values** dengan parameter nama kolom seperti pada Gambar 1.4.47 dan Gambar 1.4.48.



```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data, index=[2, 3, 0, 1])
print(df)
df_sorted = df.sort_values(by='Nama')
print(df_sorted)
```

	Nama	Umur	Alamat	Pekerjaan
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin
0	Dwi	34	Solo	Web developer
1	Andi	23	Jakarta	Programmer

	Nama	Umur	Alamat	Pekerjaan
1	Andi	23	Jakarta	Programmer
0	Dwi	34	Solo	Web developer
2	Jack	20	Surabaya	Programmer
3	Joko	23	Jakarta	DB Admin

Gambar 1.4.47 Sorting Data Berdasarkan Kolom Tertentu

```
df_sorted = df.sort_values(by=['Alamat', 'Nama'])
df_sorted
```

	Nama	Umur	Alamat	Pekerjaan
1	Andi	23	Jakarta	Programmer
3	Joko	23	Jakarta	DB Admin
0	Dwi	34	Solo	Web developer
2	Jack	20	Surabaya	Programmer

Gambar 1.4.48 Sorting Data Berdasarkan Kolom Tertentu

10) Filtering

Filtering dataset dilakukan dengan memberikan kondisi logika pada *DataFrame* seperti pada Gambar 1.4.49.

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data)
print(df)
filtered_df = df[df.Umur > 30]
filtered_df
```

	Nama	Umur	Alamat	Pekerjaan
0	Jack	20	Surabaya	Programmer
1	Joko	23	Jakarta	DB Admin
2	Dwi	34	Solo	Web developer
3	Andi	23	Jakarta	Programmer

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer

Gambar 1.4.49 Filtering Data dengan Kondisi

Filtering juga dapat dilakukan menggunakan lebih dari satu kondisi seperti pada Gambar 1.4.50.

```
filtered_df = df[(df.Umur > 20) & (df.Alat== 'Jakarta')]
filtered_df
```

	Nama	Umur	Alamat	Pekerjaan
1	Joko	23	Jakarta	DB Admin
3	Andi	23	Jakarta	Programmer

Gambar 1.4.50 Filtering Data dengan Kondisi

Filtering juga dapat dilakukan menggunakan fungsi `isin()`, yang mengecek apakah nilai pada kolom termasuk dalam sebuah list tertentu.

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer']}
df = pd.DataFrame(data)
nama1 = ['Dwi', 'Andi']
filtered_df = df[df.Nama.isin(nama1)]
filtered_df
```

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer
3	Andi	23	Jakarta	Programmer

Gambar 1.4.51 Filtering Data dengan Fungsi isin()

```
filtered = df.query("Nama=='Joko' | Nama=='Andi'")
filtered
```

	Nama	Umur	Alamat	Pekerjaan
1	Joko	23	Jakarta	DB Admin
3	Andi	23	Jakarta	Programmer

Gambar 1.4.52 Filtering Data dengan Query

Filtering dapat dilakukan menggunakan fungsi **query()**, yang memungkinkan penulisan kondisi logis dalam bentuk ekspresi *string*.

```
df.loc[(df.Umur > 30)]
```

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer

Gambar 1.4.53 Filtering Data dengan Query

Filtering juga dapat dilakukan menggunakan fungsi **str.contains()**.

```
df[df.Pekerjaan.str.contains('Web')]
```

	Nama	Umur	Alamat	Pekerjaan
2	Dwi	34	Solo	Web developer

Gambar 1.4.54 Filtering Data dengan fungsi str.contains()

11) Grouping


```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer'],
        'Gaji' : [2000, 3000, 2400, 5000]}
df = pd.DataFrame(data)
df
```

	Nama	Umur	Alamat	Pekerjaan	Gaji
0	Jack	20	Surabaya	Programmer	2000
1	Joko	23	Jakarta	DB Admin	3000
2	Dwi	34	Solo	Web developer	2400
3	Andi	23	Jakarta	Programmer	5000

Gambar 1.4.55 Membuat Dataframe

```
df.groupby('Pekerjaan').groups
```

```
{'DB Admin': [1], 'Programmer': [0, 3], 'Web developer': [2]}
```

Gambar 1.4.56 Menggabungkan Dataframe

Fungsi **groupby()** digunakan untuk mengelompokkan data berdasarkan nilai pada kolom *Pekerjaan*. Atribut **.groups** menampilkan indeks baris yang termasuk dalam setiap kelompok, sehingga terlihat pembagian data sesuai kategori pekerjaan.

```
df_group = df.groupby('Pekerjaan')
for pekerjaan, group in df_group:
    print(pekerjaan)
    print(group)
```

```
DB Admin
  Nama  Umur  Alamat  Pekerjaan  Gaji
1  Joko   23  Jakarta  DB Admin  3000
Programmer
  Nama  Umur  Alamat  Pekerjaan  Gaji
0  Jack   20  Surabaya  Programmer  2000
3  Andi   23  Jakarta  Programmer  5000
Web developer
  Nama  Umur  Alamat  Pekerjaan  Gaji
2  Dwi   34   Solo   Web developer  2400
```

Gambar 1.4.57 Menampilkan Data Menurut Pekerjaan

Objek hasil **groupby()** diiterasi untuk menampilkan setiap kelompok data secara terpisah. Variabel *pekerjaan* berisi nama kelompok, sedangkan *group* berisi *DataFrame* kecil yang memuat baris-baris sesuai kategori tersebut.



```
print(df_group.get_group('DB Admin'))
```

	Nama	Umur	Alamat	Pekerjaan	Gaji
1	Joko	23	Jakarta	DB Admin	3000

Gambar 1.4.58 Menampilkan Satu Kategori Tertentu

Fungsi `get_group()` digunakan untuk mengambil satu kelompok tertentu dari hasil `groupby()`

12) Aggregation

Operasi *aggregate* dilakukan dengan menerapkan fungsi seperti *mean*, *sum*, *min*, atau *max* pada data yang telah dikelompokkan menggunakan `groupby()`.

```
data = {'Nama': ['Jack', 'Joko', 'Dwi', 'Andi'],
        'Umur' : [20, 23, 34, 23],
        'Alamat' : ['Surabaya', 'Jakarta', 'Solo', 'Jakarta'],
        'Pekerjaan' : ['Programmer', 'DB Admin', 'Web developer', 'Programmer'],
        'Gaji' : [2000, 3000, 2400, 5000]}
df = pd.DataFrame(data)
print('sebelum di kelompokkan')
print(df)
df_group = df.groupby('Pekerjaan').mean(numeric_only=True)
print('setelah dikelompokkan')
df_group
```

```
sebelum di kelompokkan
  Nama  Umur  Alamat  Pekerjaan  Gaji
0  Jack   20  Surabaya  Programmer  2000
1  Joko   23   Jakarta    DB Admin  3000
2  Dwi   34    Solo  Web developer  2400
3  Andi   23   Jakarta  Programmer  5000
```

```
setelah dikelompokkan
      Umur  Gaji
Pekerjaan
DB Admin  23.0  3000.0
Programmer  21.5  3500.0
Web developer  34.0  2400.0
```

Gambar 1.4.59 Aggregation

Operasi *aggregate* dilakukan dengan mengelompokkan data berdasarkan kolom *Pekerjaan* menggunakan `groupby()`, kemudian menerapkan fungsi `mean()` untuk menghitung rata-rata pada kolom numerik seperti pada Gambar 1.4.60.

```
df_group = df.groupby('Pekerjaan').mean(numeric_only=True)
print('setelah dikelompokkan')
df_group
```

setelah dikelompokkan

	Umur	Gaji
Pekerjaan		
DB Admin	23.0	3000.0
Programmer	21.5	3500.0
Web developer	34.0	2400.0

Gambar 1.4.60 Aggregation

Contoh operasi *aggregate* lain dengan menerapkan beberapa fungsi secara sekaligus melalui `agg()`, setelah data dikelompokkan berdasarkan kolom *Pekerjaan*. Pada contoh ini, fungsi *mean*, *sum*, dan *std* digunakan untuk menghitung rata-rata, total, dan standar deviasi pada kolom Umur dan Gaji untuk setiap kelompok pekerjaan.

```
df_group = df.groupby('Pekerjaan').agg({
    'Umur': [np.mean, np.sum, np.std],
    'Gaji': [np.mean, np.sum, np.std]
})
print('setelah dikelompokkan')
df_group
```

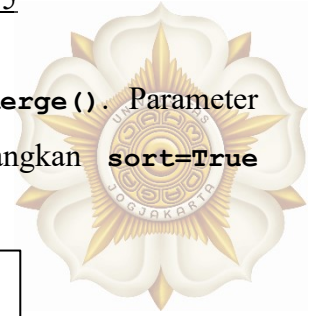
setelah dikelompokkan

/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`
/tmp/ipython-input-2583672168.py:1: FutureWarning: The provided callable `df_group = df.groupby('Pekerjaan').agg({`

	Umur			Gaji		
	mean	sum	std	mean	sum	std
Pekerjaan						
DB Admin	23.0	23	NaN	3000.0	3000	NaN
Programmer	21.5	43	2.12132	3500.0	7000	2121.320344
Web developer	34.0	34	NaN	2400.0	2400	NaN

Gambar 1.4.61 Aggregation

13) Merge



Penggabungan *dataframe* dilakukan menggunakan fungsi `merge()`. Parameter `on='Id'` menentukan *key column* yang digunakan, sedangkan `sort=True` mengurutkan hasil berdasarkan *key value* tersebut.

```
kiri = pd.DataFrame({'Id':[2, 3, 1],
'Nama' : ['Andi', 'Joko', 'Budi']})
kanan = pd.DataFrame({'Id':[1, 2, 3],
'Nama' : ['Mike', 'Jack', 'Jane']})
gabung = pd.merge(kiri, kanan, on='Id', sort=True)
gabung
```

	Id	Nama_x	Nama_y
0	1	Budi	Mike
1	2	Andi	Jack
2	3	Joko	Jane

Gambar 1.4.62 Merging Dataframe

```
karyawan = pd.DataFrame({
'Nama': ['Budi', 'Joko', 'Maya'],
'Alamat': ['Jakarta', 'Surabaya', 'Medan'],
'Id Pekerjaan' : [1,7,2]
})
pekerjaan = pd.DataFrame({
'Id Pekerjaan': [2,1,4,3],
'Nama': ['Programmer', 'Data Engineer', 'Manager', 'Web Developer'],
'Gaji' : [5000, 4000, 3000, 6000]
})
print(karyawan)
print(pekerjaan)
```

	Nama	Alamat	Id Pekerjaan
0	Budi	Jakarta	1
1	Joko	Surabaya	7
2	Maya	Medan	2

	Id Pekerjaan	Nama	Gaji
0	2	Programmer	5000
1	1	Data Engineer	4000
2	4	Manager	3000
3	3	Web Developer	6000

Gambar 1.4.63 Merging Dataframe

Penggabungan juga dapat dilakukan menggunakan `merge()` dengan parameter `how='left'`, sehingga semua baris dari *DataFrame* karyawan tetap ditampilkan, sementara data dari *DataFrame* pekerjaan hanya ditambahkan jika nilai Id Pekerjaan cocok.

```
gabung_merge = pd.merge(karyawan, pekerjaan, on='Id Pekerjaan',
                        how='left', sort=True)
gabung_merge
```

	Nama_x	Alamat	Id Pekerjaan	Nama_y	Gaji
0	Budi	Jakarta	1	Data Engineer	4000.0
1	Maya	Medan	2	Programmer	5000.0
2	Joko	Surabaya	7	NaN	NaN

Gambar 1.4.64 Merging Dataframe

14) Join

Join dapat dilakukan dengan menggunakan **merge()** pada *key column* yang sama, di mana parameter **how='left'** memastikan seluruh data dari tabel kiri ditampilkan, sedangkan *suffixes* digunakan untuk membedakan nama kolom yang berasal dari kedua *DataFrame*.

```
#menggunakan suffixes
gabung_merge = pd.merge(karyawan, pekerjaan, on='Id Pekerjaan',
                        how='left', sort=True, suffixes=('_pegawai', '_pekerjaan'))
gabung_merge
```

	Nama_pegawai	Alamat	Id Pekerjaan	Nama_pekerjaan	Gaji
0	Budi	Jakarta	1	Data Engineer	4000.0
1	Maya	Medan	2	Programmer	5000.0
2	Joko	Surabaya	7	NaN	NaN

Gambar 1.4.65 Joining Dataframe

Join juga dapat dilakukan dengan fungsi **join()**. *Join* bekerja berdasarkan indeks, sehingga hasilnya bisa berbeda dari **merge()** yang menggunakan *key column*. Parameter **how='left'** menjaga seluruh baris *DataFrame* kiri tetap tampil, sementara *rsuffix* digunakan untuk membedakan kolom yang duplikat dari *DataFrame* kanan.

```
#versi 1. hasil tidak sama dengan merge()
gabung_join = karyawan.join(pekerjaan, how='left', rsuffix='_pekerjaan')
gabung_join
```

	Nama	Alamat	Id Pekerjaan	Id Pekerjaan_pekerjaan	Nama_pekerjaan	Gaji
0	Budi	Jakarta	1	2	Programmer	5000
1	Joko	Surabaya	7	1	Data Engineer	4000
2	Maya	Medan	2	4	Manager	3000

Gambar 1.4.66 Joining Dataframe

15) Concatenate

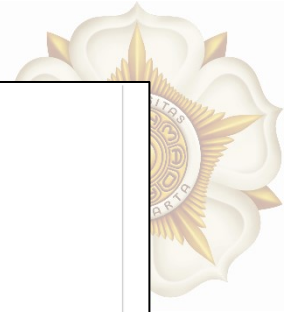
Concatenate dilakukan dengan menggunakan `pd.concat()` pada `axis=0`, sehingga *DataFrame* kedua ditambahkan sebagai baris baru di bawah *DataFrame* pertama. Parameter `ignore_index=True` digunakan agar indeks diurutkan ulang secara otomatis.

```
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Nama': ['Jane', 'mike', 'dave'],
    'Alamat': ['Semarang', 'Yogya', 'Solo']
})
gabung_concat = pd.concat([kiri, kanan], axis=0, ignore_index=True)
gabung_concat
```

	Nama	Alamat
0	Jane	Semarang
1	mike	Yogya
2	dave	Solo
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 1.4.67 Concatenate

Dengan menambahkan parameter `axis=0` dan parameter `join='outer'` dapat menggabungkan semua kolom dari kedua *DataFrame*. Kolom yang tidak dimiliki oleh salah satu *DataFrame* akan berisi nilai *NaN*, sementara `ignore_index=True` mengatur ulang indeks agar berurutan.



```
# menjadi baris baru. axis=0
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Nama': ['Jane', 'mike', 'dave'],
    'Gaji': [5000, 4000, 3000]
})
gabung_concat = pd.concat([kiri, kanan], axis=0, ignore_index=True, join='outer')
gabung_concat
```

	Nama	Alamat	Gaji
0	Budi	Jakarta	NaN
1	Joko	Surabay	NaN
2	Maya	Medan	NaN
3	Jane	NaN	5000.0
4	mike	NaN	4000.0
5	dave	NaN	3000.0

Gambar 1.4.68 Concatenate

Jika menggunakan **axis=1**, penggabungan dilakukan secara horizontal sehingga kolom baru ditambahkan di samping *DataFrame* awal.

```
#gabung menjadi kolom baru. axis =1
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Pekerjaan': ['Programmer', 'Data Engineer', 'Manager', 'Web Developer'],
    'Gaji': [5000, 4000, 3000, 6000]
})
gabung_concat = pd.concat([kiri, kanan], axis=1, join='outer')
gabung_concat
```

	Nama	Alamat	Pekerjaan	Gaji
0	Budi	Jakarta	Programmer	5000
1	Joko	Surabay	Data Engineer	4000
2	Maya	Medan	Manager	3000
3	NaN	NaN	Web Developer	6000

Gambar 1.4.69 Concatenate

16) Reset Indeks



```
kiri = pd.DataFrame({
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat': ['Jakarta', 'Surabay', 'Medan']
})
kanan = pd.DataFrame({
    'Nama': ['Jane', 'mike', 'dave'],
    'Alamat': ['Semarang', 'Yogya', 'Solo']
})
gabung_append = pd.concat([kiri, kanan], ignore_index=True)
gabung_append
```

	Nama	Alamat
0	Budi	Jakarta
1	Joko	Surabay
2	Maya	Medan
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 1.4.70 Reset Index

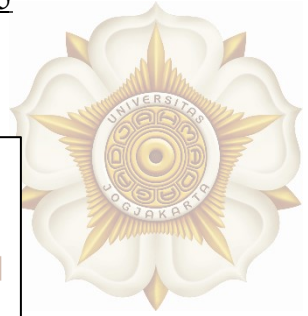
Reset *index* dilakukan dengan menggunakan `pd.concat()` dan menetapkan `ignore_index=True`, sehingga indeks lama dari kedua *DataFrame* diabaikan dan diganti dengan indeks baru yang berurutan setelah proses penggabungan.

```
new = gabungan_append.reset_index(drop=True)
new
```

	Nama	Alamat
0	Budi	Jakarta
1	Joko	Surabay
2	Maya	Medan
3	Jane	Semarang
4	mike	Yogya
5	dave	Solo

Gambar 1.4.71 Reset Index

Reset indeks juga dapat dilakukan dengan fungsi `reset_index(drop=True)`, yang menghapus indeks lama dan menggantinya dengan indeks baru yang berurutan. Parameter `drop=True` memastikan indeks sebelumnya tidak ikut disimpan sebagai kolom baru.



17) Set Indeks

```
kiri = pd.DataFrame({
    'ID' : ['001', '002', '003'],
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Alamat' : ['Jakarta', 'Surabay', 'Medan']
})
print(kiri)
```

	ID	Nama	Alamat
0	001	Budi	Jakarta
1	002	Joko	Surabay
2	003	Maya	Medan

Gambar 1.4.72 Set Index

```
kiri.iloc[2]
```

2

ID 003

Nama Maya

Alamat Medan

dtype: object

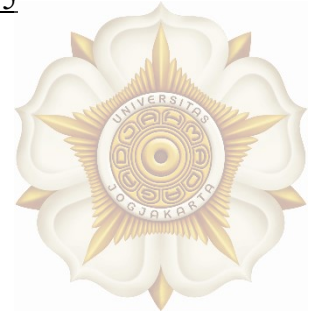
Gambar 1.4.73 Set Index

Set indeks dapat dilakukan dengan memilih baris tertentu sebagai referensi indeks baru. Pada Gambar 1.4.73, nilai pada baris ke-2 ditampilkan karena *DataFrame* sebelumnya telah atau akan diubah indeksnya menggunakan nilai kolom tertentu melalui `set_index()`, sehingga baris tersebut ditampilkan sesuai indeks barunya.

```
kiri.set_index('ID', inplace=True)
kiri
```

	Nama	Alamat
ID		
001	Budi	Jakarta
002	Joko	Surabay
003	Maya	Medan

Set indeks juga dapat dilakukan dengan fungsi `set_index()` yang menetapkan kolom *ID* sebagai indeks baru *DataFrame*.



```
kiri.loc['003']
```

	003
Nama	Maya
Alamat	Medan

```
dtype: object
```

Gambar 1.4.74 Set Index

`kiri.loc['003']` menampilkan data dengan indeks bernilai '003', sesuai indeks baru yang telah ditetapkan menggunakan kolom *ID*.

18) Pivot Tabel

```
karyawan = pd.DataFrame({
    'Emp_id' : ['001', '002', '003', '004', '005', '006'],
    'Nama': ['Budi', 'Joko', 'Maya', 'Andi', 'Dewi', 'Dian'],
    'Gender': ['Pria', 'Pria', 'Perempuan', 'Pria', 'Perempuan', 'Perempuan'],
    'Pekerjaan': ['Programmer', 'Data Engineer', 'Programmer', 'Manager', 'Manager', 'Manager'],
    'Gaji': [5000, 4000, 5000, 6000, 8000, 6000]})
karyawan
```

	Emp_id	Nama	Gender	Pekerjaan	Gaji
0	001	Budi	Pria	Programmer	5000
1	002	Joko	Pria	Data Engineer	4000
2	003	Maya	Perempuan	Programmer	5000
3	004	Andi	Pria	Manager	6000
4	005	Dewi	Perempuan	Manager	8000
5	006	Dian	Perempuan	Manager	6000

Gambar 1.4.75 Membuat Dataframe

Pivot tabel dibuat menggunakan `pd.pivot_table()` dengan menetapkan kolom *Gender* dan *Pekerjaan* sebagai pembentuk kolom baru. Agregasi dilakukan melalui parameter `aggfunc`, untuk menjumlahkan nilai Gaji dan menghitung jumlah data berdasarkan Pekerjaan seperti pada Gambar 1.4.76

```

pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': 'sum', 'Pekerjaan': 'count'})
pivot_table

```

Gender	Perempuan	Pria
Pekerjaan	Manager	Programmer
Gaji	14000	5000
Pekerjaan	2	1

Gambar 1.4.76 Pivot Table

Menghitung sum, *mean*, median, dan lain-lain dapat dilakukan dengan `np.sum` seperti pada Gambar 1.4.77.

```
#cara kedua pakai numpy untuk menghitung sum, mean, dan lain lain
import numpy as np
pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': np.sum, 'Pekerjaan': 'count'})
pivot_table
```

/tmp/ipython-input-2104525659.py:3: FutureWarning: The provided callable <function sum at 0x7c2825b73100> is currently u
pivot_table = pd.pivot_table(karyawan, columns=['Gender', 'Pekerjaan'], aggfunc={'Gaji': np.sum, 'Pekerjaan': 'count'})

Gender	Perempuan	Pria			
Pekerjaan	Manager	Programmer	Data Engineer	Manager	Programmer
Gaji	14000	5000	4000	6000	5000
Pekerjaan	2	1	1	1	1

Gambar 1.4.77 Pivot Table

19) Melt

Melt digunakan untuk mengubah *DataFrame* dari format lebar (*wide*) menjadi format panjang (*long*).

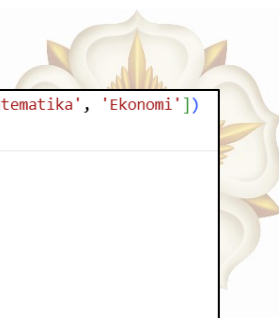
```

mahasiswa = pd.DataFrame({
    'Mhs_id': ['001', '002', '003'],
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Bhs Inggris': [100, 90, 98],
    'Matematika': [90, 87, 72],
    'Ekonomi': [90, 80, 78]})
mahasiswa

```

	Mhs_id	Nama	Bhs Inggris	Matematika	Ekonomi
0	001	Budi	100	90	90
1	002	Joko	90	87	80
2	003	Maya	98	72	78

Gambar 1.4.78 Membuat Dataframe



```
melt_table = pd.melt(mahasiswa, id_vars=['Mhs_id', 'Nama'], value_vars=['Bhs Inggris', 'Matematika', 'Ekonomi'])
melt_table
```

	Mhs_id	Nama	variable	value
0	001	Budi	Bhs Inggris	100
1	002	Joko	Bhs Inggris	90
2	003	Maya	Bhs Inggris	98
3	001	Budi	Matematika	90
4	002	Joko	Matematika	87
5	003	Maya	Matematika	72
6	001	Budi	Ekonomi	90
7	002	Joko	Ekonomi	80
8	003	Maya	Ekonomi	78

Gambar 1.4.79 Melt Table

Kolom yang ditentukan pada **id_vars** tetap dipertahankan, sedangkan kolom pada **value_vars** diubah menjadi dua kolom baru, yaitu *variable* dan *value* sehingga tiap nilai mata pelajaran ditampilkan sebagai baris terpisah.

20) Lambda Function

Berikut penggunaan fungsi lambda dengan dua parameter untuk melakukan operasi penjumlahan.

```
x = lambda var1, var2 : var1 + var2
hasil = x(10, 12)
hasil
```

22

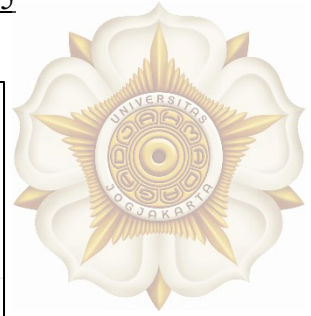
Gambar 1.4.80 Lambda Function

Berikut penggunaan fungsi lambda dengan satu parameter untuk melakukan operasi perkalian.

```
test = lambda var1 : var1 * 2
hasil = test(5)
hasil
```

10

Gambar 1.4.81 Lambda Function



```
mahasiswa = pd.DataFrame({
    'Mhs_id' : ['001', '002', '003'],
    'Nama': ['Budi', 'Joko', 'Maya'],
    'Matematika': [90, 47, 72]})
mahasiswa
```

	Mhs_id	Nama	Matematika
0	001	Budi	90
1	002	Joko	47
2	003	Maya	72

Gambar 1.4.82 Lambda Function

Berikut penggunaan fungsi lambda dengan kondisi bertingkat untuk menentukan nilai berdasarkan skor Matematika pada *DataFrame*.

```
mahasiswa['Grade'] = mahasiswa['Matematika'].apply(lambda nilai :
    'A' if nilai >=90 else 'B' if nilai >=70 and nilai <90 else ('C'))
mahasiswa
```

	Mhs_id	Nama	Matematika	Grade
0	001	Budi	90	A
1	002	Joko	47	C
2	003	Maya	72	B

Gambar 1.4.83 Lambda Function

21) Membaca File

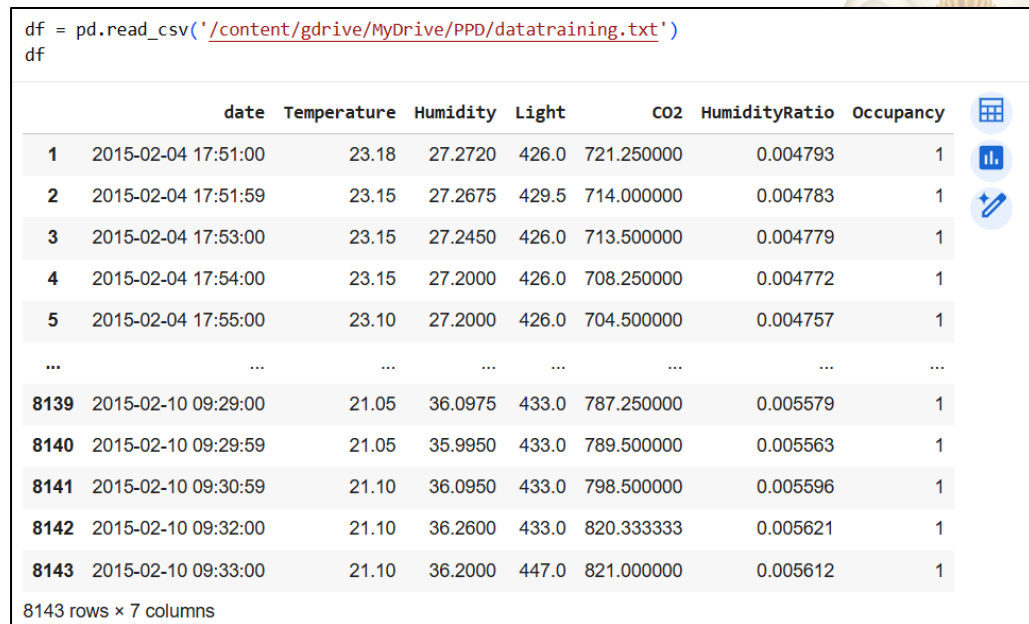
Membaca *file* dari Google Drive dapat dilakukan dengan menggunakan `drive.mount()`.

```
from google.colab import drive
drive.mount('/content/gdrive/')

Mounted at /content/gdrive/
```

Gambar 1.4.84 Menghubungkan Google Colaboratory dengan Google Drive

Lalu untuk membaca *file* dataset dari Google Drive, dapat dilakukan dengan menggunakan `pd.read_csv()` seperti pada Gambar 1.4.85.



```
df = pd.read_csv('/content/gdrive/MyDrive/PPD/datatraining.txt')
df
```

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.250000	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.000000	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.500000	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.250000	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.500000	0.004757	1
...	...	...	...	...	...	...	...
8139	2015-02-10 09:29:00	21.05	36.0975	433.0	787.250000	0.005579	1
8140	2015-02-10 09:29:59	21.05	35.9950	433.0	789.500000	0.005563	1
8141	2015-02-10 09:30:59	21.10	36.0950	433.0	798.500000	0.005596	1
8142	2015-02-10 09:32:00	21.10	36.2600	433.0	820.333333	0.005621	1
8143	2015-02-10 09:33:00	21.10	36.2000	447.0	821.000000	0.005612	1

8143 rows × 7 columns

Gambar 1.4.85 Membaca File dari Google Drive

1.5. TUGAS & ANALISIS

1. Buatlah *array* NumPy 2D berukuran 4×4 berisi angka acak (0–50), kemudian ubah bentuknya menjadi *array* 2×8 dan tampilkan hanya elemen-elemen yang bernilai lebih dari 25.
2. Buatlah *dataframe* untuk data karyawan yang berisi kolom Nama, Pekerjaan, dan Gaji. Lalu lakukan operasi **groupby()** pada kolom Pekerjaan dan hitung nilai maksimum, minimum, serta standar deviasi untuk kolom ‘Gaji’ menggunakan **agg()**.
3. Buat dua *DataFrame* untuk data karyawan dengan kolom Emp_id, Nama, dan Divisi, serta *DataFrame* gaji dengan kolom ID, Gaji, dan Status (masing-masing minimal tiga baris). Setelah itu, gabungkan kedua *DataFrame* tersebut menggunakan **merge()** dengan **left_on='Emp_id'** dan **right_on='ID'**, terapkan *left join*, dan gunakan *suffixes* untuk membedakan kolom yang sama.

1.6. REFERENSI

<https://numpy.org/>

<https://pandas.pydata.org/>

<https://www.python.org/doc/>